Northwind Logistics Delivery-Tracking Service
DevOps Technical Report

CSO7024 - DevOps Final Project

Author: Shahzad Sadruddin
Student ID: 2513806

Word count (sections a-d): 1450

a) Solution architecture and toolchain

Northwind Logistics's delivery-tracking service is a small read-only HTTP API (Python 3 standard-library http.server, mirrored by a Go implementation for comparison) exposing /health, /metrics, /deliveries and /deliveries/{id}. Around this application I built an integrated toolchain covering all six DevOps disciplines: Git and GitHub for version control, pytest and Go's testing package for automated tests, GitHub Actions for CI/CD, Terraform together with Ansible for environment automation, and Docker with Kubernetes for containerisation and orchestration.

Git and GitHub host the single source of truth; every change is committed directly with a descriptive, conventional-commit-style message (feat:, fix:, ci:, test:, docs:) so the resulting history reads as a narrative of the build (Exhibit 2). GitHub Actions was chosen over GitLab CI or Jenkins because the repository is already hosted on GitHub, giving native integration with the GitHub Container Registry (GHCR) and free hosted runners, plus a self-hosted runner for the final Kubernetes deploy step. pytest was chosen for the Python suite for its fixtures and parametrisation; Go's built-in testing package plus testify mirrors the same behaviour for the comparison service, so both languages are tested with idiomatic, native tools rather than forcing one framework onto both.

For environment automation I used both permitted approaches for different targets rather than picking one for the whole system: Terraform provisions the AWS infrastructure (VPC, EC2 host, RDS PostgreSQL instance, ECR repository, IAM role) because provisioning cloud resources is naturally declarative; Ansible then configures that EC2 host (installing Docker, templating a Compose file, starting the container) because configuring an existing machine is imperative and step-based, which fits Ansible's task model better than restating it inside Terraform's user_data script. Docker packages both services as minimal, non-root images (a slim CPython image for Python, a two-stage distroless build for Go); Kubernetes, via a local kind/minikube cluster, runs two replicas of the app behind a Service, backed by a PostgreSQL Deployment, with readiness and liveness probes and a smoke-test Job.

Exhibit 1 shows the flow: a commit to main triggers lint, then pytest (with a Postgres service container), then Go vet/test, then build and push of both images to GHCR, then a smoke test of the Python container, then deployment to the local kind cluster via a self-hosted runner. The same GHCR image is what Ansible pulls onto the Terraform-provisioned EC2 host, so the cloud and Kubernetes paths converge on one artefact rather than each environment building its own copy.

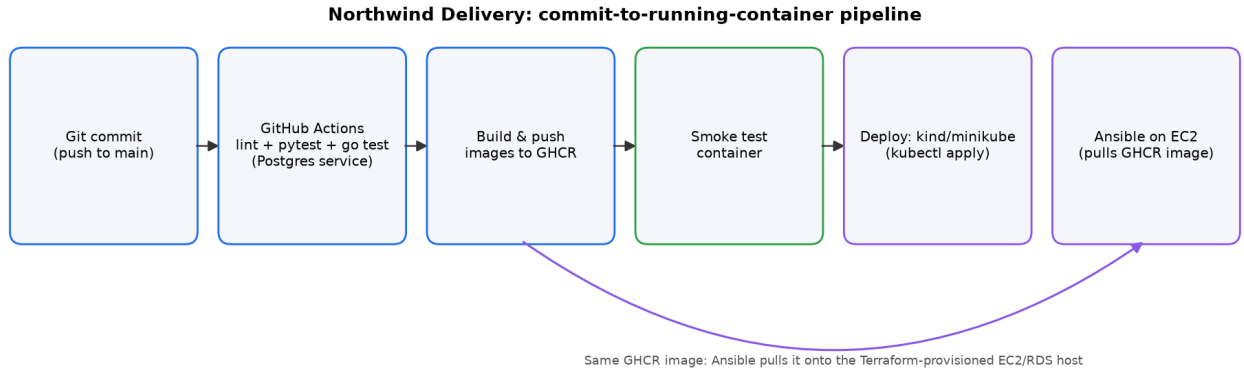

Exhibit 1: commit-to-running-container pipeline flow.

Learning outcome	Tool used	Evidence (path)
LO3 - Version control	Git + GitHub	git log; conventional commit messages

Learning outcome	Tool used	Evidence (path)
LO4 - Automated testing	pytest, Go testing + testify	tests/test_app.py; go/pkg/**/*_test.go
LO4 - CI/CD	GitHub Actions	.github/workflows/ci-cd.yml
LO5 - Configuration/IaC	Terraform + Ansible	terraform/*.tf; ansible/playbook.yml
LO6 - Containers/orchestration	Docker + Kubernetes	Dockerfile; go.Dockerfile; k8s/*.yaml

Exhibit 2: DevOps learning outcomes mapped to tool and evidence.

b) Implementation and evidence

Each rubric area has a concrete home in the repository. Version control: the full history is visible with git log, with 26 commits on main; because this is an individually assessed project I did not open pull requests, but every commit message documents intent and most map to one reviewable change (discussed further in section d). Automated testing: tests/test_app.py contains 20 test classes covering seed data, filtering, counting, every HTTP endpoint, response headers, concurrency, and Postgres-backed integration and performance behaviour — 70 test functions in total, many parametrised; go/pkg/data/data_test.go and go/pkg/handlers/handlers_test.go add 23 and 27 Go tests respectively, including table-driven cases and concurrent-request tests run against a real net/http server (Exhibit 3).

CI/CD: .github/workflows/ci-cd.yml defines six jobs — lint, test, test-go, build, smoke-test, deploy — each depending on the previous one succeeding, so a failing test blocks the build and deploy stages (Exhibit 4). The test and test-go jobs run against a real postgres:16-alpine service container, so the Postgres-backed code paths are exercised in CI, not only the in-memory/SQLite fallback used for fast local runs. Every run publishes JUnit XML and an HTML pytest report as artefacts, and scripts/generate_dashboard.py turns those into a persistent HTML dashboard (dashboard/index.html) with a run-history log — my evidence of at least one successful end-to-end pipeline run.

Configuration/IaC: terraform/ec2.tf and terraform/rds.tf provision the VPC, EC2 host and RDS instance idempotently (re-running terraform apply with no underlying changes reports "No changes" rather than duplicating resources); ansible/playbook.yml uses idempotent modules (dnf, systemd, template) so repeated runs converge rather than redoing work, and finishes with live smoke tests against /health and /deliveries on the deployed host.

Containers/orchestration: Dockerfile and go.Dockerfile build non-root images with HEALTHCHECK instructions; k8s/deployment.yaml runs 2 replicas with readiness and liveness probes against /health, k8s/service.yaml exposes the app as a LoadBalancer, and k8s/deploy.sh applies everything in dependency order before running k8s/smoke-test.yaml as a final reachability check. A marker can reproduce this by running "docker compose up --build" locally, "pytest tests -q" and "go test ./..." for the tests, and "./k8s/deploy.sh" against a running kind or minikube cluster.

Suite	Files	Test functions	Notable coverage
Python (pytest)	tests/test_app.py	70 (20 classes)	endpoints, filtering, concurrency, Postgres integration, latency
Go (data)	go/pkg/data/data_test.go	23	table-driven, concurrent create/update, clone isolation
Go (handlers)	go/pkg/handlers/handlers_test.go	27	pagination, sorting, request IDs, concurrent real-server reque
Go (store)	go/pkg/store/postgres_test.go	1	Postgres-backed store integration

Exhibit 3: automated test suite summary (Python and Go).

Job	Depends on	Purpose
lint	-	ruff static analysis of the Python code
test	lint	pytest suite against SQLite + a Postgres service container
test-go	-	go vet and go test -race -cover against a Postgres service container
build	test, test-go	build & push Python and Go images to GHCR
smoke-test	build	run the built container and curl /health, /deliveries
deploy	smoke-test	self-hosted runner deploys to the local kind cluster (main only)

Exhibit 4: GitHub Actions pipeline jobs and dependencies.

c) Critical evaluation and trade-offs

The strongest part of the solution is its integration: one GHCR image is the single artefact consumed by both the Kubernetes deployment and the Ansible-configured EC2 host, so "build once, deploy everywhere" is genuinely true rather than aspirational. Running the same tests against both an in-memory/SQLite store and a real Postgres service container in CI also gives real confidence, rather than only a syntactic pass.

The main weakness is version control practice: because this is an individual assessment, I committed directly to main rather than using feature branches and pull requests, so the "reviewable by a teammate" bar is met only through descriptive commit messages, not an actual review workflow — in a team I would use short-lived feature branches merged via PRs with required status checks before merge. A second trade-off is scope: the application itself is deliberately left almost unchanged, per the brief, so the engineering effort sits entirely in the surrounding automation; this keeps risk low but means the DevOps value is best judged by the pipeline, not by new application features.

Security-wise, the RDS instance is not publicly reachable (its security group is scoped to the EC2 host's security group only), secrets are Ansible-Vault-encrypted, and containers run as non-root with HEALTHCHECKs — but the Kubernetes Postgres credentials in k8s/postgres.yaml are still placeholder values that must be replaced before any shared or production use, and the Terraform SSH ingress rule defaults to 0.0.0.0/0 unless a narrower CIDR is supplied, which I flag rather than hide. Reliability-wise, a single EC2 instance and a single-AZ RDS instance are cost-appropriate for coursework but are single points of failure; the Kubernetes side is more resilient with two replicas and self-healing via probes, but still runs on one local node, so it does not survive a node failure.

Given more time I would add a staging/production split using Terraform workspaces, adopt real GitOps-style promotion (for example Argo CD watching the GHCR tag) instead of the pipeline calling kubectl directly, and exercise a Create/UpdateStatus write path end to end to prove the shared-Postgres design under real writes, not just reads.

d) Professional practice, industry currency and reflection

The toolchain mirrors current industry norms rather than novel choices: GitHub Actions and GHCR are among the most widely used CI/CD-and-registry pairings for GitHub-hosted projects, container health checks and Kubernetes readiness/liveness probes are the standard mechanism for zero-downtime rollouts, and separating a declarative provisioning tool (Terraform) from an imperative configuration tool (Ansible) reflects the still-common "IaC provisions, configuration management configures" pattern rather than treating either as a one-size-fits-all solution.

Two current trends this project touches but does not fully adopt are GitOps and DORA metrics. GitOps — where a controller such as Argo CD or Flux continuously reconciles a cluster against a Git repository rather than a pipeline pushing changes imperatively — is an increasingly recommended pattern for Kubernetes deployment, reflected in the CNCF's own project landscape and its annual cloud-native survey; my deploy job still calls "kubectl apply" directly from the pipeline, which is simpler to reason about for a single coursework environment but would not scale cleanly to multiple clusters. Second, Google Cloud's DORA "State of DevOps" research popularised four key delivery metrics — deployment frequency, lead time for changes, change failure rate and time to restore — as the standard way teams measure DevOps maturity; my dashboard's persistent run-history log is a small step in that direction (it tracks per-run pass/fail and timing), but a mature implementation would compute these metrics automatically from the Git and Actions APIs rather than from a bespoke JSON log.

On collaboration and communication: working individually meant I could not practise real code review, but I still wrote every commit message as if a teammate would read it — stating what changed and why, for example "ci: remove AWS ECR push from workflow to avoid credential errors" — and kept the README and this report as the artefacts a new team member would actually need to get productive. If I were joining a team using this repository, the biggest adjustment would be moving from direct-to-main commits to short-lived feature branches with mandatory pull request review and required CI status checks before merge, which the pipeline as designed would support with only a branch-protection-rule change, since it already runs on both push and pull_request events.